

MacroFit Final Project Report

Nagur Pavan | PE6201 Emerging AI Technologies | 1 October 2026

Purpose This report explains the final product, the reasoning behind its hybrid architecture, the evidence obtained, and the limitations that remain before real-world use.

Executive Summary

MacroFit is a notebook-based meal planner for adults who depend on prepared food. A user supplies a profile, budget, and ordinary food request. Python estimates general nutrition and builds a valid breakfast, lunch, and dinner shortlist; a rented language model interprets subjective preferences and selects within that shortlist. The prototype tests whether this hybrid process produces practical choices without giving AI control over arithmetic or hard constraints.

The latest evaluation passed 19 of 20 pipeline cases and cost US\$0.00546 315. This supports the architecture, including unusual inputs, but does not yet prove model value: five blind judgements were ties. Only cases where AI and the optimizer choose different plans can answer whether the model earns its place.

Problem and Intended Difference

People who buy most meals repeatedly compare budget, nutrition, mood, and many menu items. Calorie tools mainly record food after eating, while ordering platforms rank single items without balancing the day. MacroFit combines these decisions in one bounded flow.

Users can request spicy food, vegetarian lunch, light dinner, portability, or no rice. MacroFit converts those words into constraints and preferences, schedules meals, remembers recommendations during the active session, and recalculates the remaining day after confirmed intake. Contradictory, unsafe, or infeasible requests receive an explanation rather than a misleading plan. This is a decision aid, not medical advice.

Technical and Business Reasoning

The own-or-rent boundary is decided operation by operation. I own the Mifflin St Jeor calculation, activity and goal adjustment, macro split, data join, combination search, budget check, 90 percent threshold, meal-slot rules, dietary filtering, duplicate prevention, and shortlist construction in Python. These are exact, testable operations; renting model judgement for them would reduce control without adding value. I rent only language interpretation, subjective shortlist ranking, and actual-intake interpretation. AI Layer 1 converts flexible wording about cuisine, spice, heaviness, portability, and food format into a closed schema. AI Layer 2 may choose only an exact identifier from Python's valid shortlist. Python validates every model result and recalculates nutrition and budget.

The five build-versus-buy factors support this split. Cost favours renting at prototype volume: the entire evaluation cost US\$0.00546 315, while training or hosting a foundation model would be disproportionate. Latency is acceptable for an interactive notebook because calculations remain local and only compact JSON is sent, although response time was not measured and is a current evidence gap. Control favours building all hard constraints and surrounding the rented model with schemas, an allowlist, retries, and output checks. Data gravity is low because the prototype sends a request and a maximum of five compact synthetic plans rather than the full dataset or persistent history. Regulation favours keeping auditable decisions in Python and avoiding medical claims; a real service would still require privacy, residency, provider-contract, and nutrition review.

The problem statement mentioned Claude, but the final design does not depend on Claude or any one vendor. OpenRouter makes the configured model replaceable and was chosen for low-cost access to schema-capable models. This reduces lock-in, but provider rate limits, malformed output, latency variation, and behaviour changes remain external risks. The model earns its place only if blind human comparison shows that its valid choices are preferred to the optimizer baseline; constraint compliance is not counted as an AI achievement.

Data and Responsible Use

The dummy set contains 60 menu items from 10 synthetic outlets and 60 matching nutrition rows. Each item records price, cuisine, dietary type, spice, heaviness, format, portability, and tags. Each nutrition row records calories, protein, carbohydrates, fat, fibre, sodium, source family, matching method, mapper, date, and confidence. Nagur Pavan performed the generic-dish matching and documented uncertainty: 8 mappings are high confidence, 36 medium, and 16 low.

The data validates joins and behaviour, not current prices or medical accuracy. A silent label error can pass through correct code: Egg Fried Rice was initially marked vegetarian, and Yong Tau Foo H04 still needs review because its generic match refers to fishcake. Real use requires measured portions, timestamped prices, verified dietary and allergen labels, and independent nutrition review.

Metrics and Final Outcomes

The 20 human-written cases contain 12 normal requests and 8 outliers: contradictions, unclear references, an infeasible budget, prompt attacks, a complex preference, and an extreme intake conflict. For each eligible case, the optimizer and model choose from the same shortlist; a blind judge records which plan they would actually eat. The latest run passed 19 cases, or 95 percent, using 13 Layer 1 calls, 12 Layer 2 calls, 25 API calls, 25,997 prompt tokens, and 2,606 completion tokens.

Provider-reported evaluation cost was US\$0.00546 315. US\$0.00109 263 is cost per valid judged recommendation, not per AI win. AI and the optimizer differed in 7 of 12 Layer 2 cases and matched in 5. All five entered blind judgements were ties, so cost per AI-preferred win is unavailable.

Metric	Result	Interpretation
Pipeline pass rate	19 of 20	95 percent
Layer 1 calls	13	Language interpretation
Layer 2 calls	12	Subjective shortlist ranking
Different AI selections	7 of 12	Eligible for blind comparison
Evaluation cost	US\$0.00546 315	Provider reported
Blind evidence	5 ties	No AI preference win yet

Evaluation Critique

The 95 percent pass rate is encouraging because outliers test more than the happy path. Injection attempts stopped before a model call; contradictions and unclear references requested clarification; the S\$2 case was infeasible. E06 failed because the deterministic precheck matched veg inside non-veg and reported a false contradiction. Standalone veg must not match when preceded by non-, and all cases must then be rerun.

Constraint accuracy belongs to Python and is not evidence of AI value. Identical selections should be recorded as same selection, not blind ties. Only the seven changed selections can compare preference, and five judgements are too few for a reliable conclusion. The honest result is that AI produced valid subjective choices cheaply, but current human evidence does not show that they are better.

Difficulties and Performance Tuning

The main design difficulty was stopping AI from doing work Python already solves. The final optimizer is the non-AI baseline; the model handles language and subjective preference only. Ambiguous terms such as veg, non-veg, morning, and that meal require explicit mapping or clarification. Live intake mode asks up to two follow-ups, while evaluation records clarification and continues.

Repeated recommendations exposed another weakness: a large menu still repeats when a few plans dominate the score. I enlarged and diversified the candidate pool, penalised recently eaten food, and excluded items already recommended in the session. Efficiency improved by sending at most five compact plans, limiting output tokens, skipping Layer 2 when subjective judgement is unnecessary, and retrying malformed output once.

Rough Edges and Future Path

Rough edges include synthetic data, temporary Colab history, incomplete blind evaluation, and rules needing review. The prototype does not certify allergens, connect to live menus, or support clinical diets, and the model may misunderstand unusual phrasing.

Next, I would correct E06, review H04, rerun all cases, and judge every changed selection blindly. A pilot should persist user-controlled history, add an egg preference, verify a small real menu, and log model and dataset versions. Python should retain calculations and constraints; AI should remain only if a larger blind study shows enough preference benefit to justify its cost and complexity.